

Secure Password Vault Using AES-CBC + Diffie– Hellman

Fall Semester 2025

Group No.14	
Student Name - ID	Tasks
Shayma Aljuaid	Research Part: Key Derivation Using SHA-256 (The Role of SHA-256 in Key Production), AES-CBC Symmetric Encryption, Real-World Applications.
Lujain Alqarni	Research Part: Introduction to Hybrid Cryptography, Diffie–Hellman Key Exchange, Key Derivation Using SHA-256 (Why Convert the Raw Shared Secret).

Table of Contents

Introduction to Hybrid Cryptography	3
Diffie–Hellman Key Exchange	3
Key Derivation Using SHA-256	4
AES-CBC Symmetric Encryption	5
Real-World Applications	6
Conclusion	7
References	8

Introduction to Hybrid Cryptography

Hybrid cryptography represents a robust approach to secure communication by combining the strengths of asymmetric key exchange algorithms with symmetric encryption techniques. In this framework, asymmetric algorithms, such as Diffie-Hellman (DH) or Elliptic Curve Diffie-Hellman (ECDH), are used to securely establish a shared session key between communicating parties over an insecure channel (Deokar, 2022). This process ensures that the session key is only accessible to legitimate participants, minimizing the risk of interception or misuse during transmission (Deokar, 2022).

Once the session key is securely established, symmetric encryption algorithms, most commonly AES (Advanced Encryption Standard), are employed to encrypt the actual message data efficiently (Deokar, 2022). The use of symmetric encryption provides high-speed data processing and reduces computational overhead compared to purely asymmetric systems, making it suitable for real-time communication and resource-constrained environments (Deokar, 2022).

Overall, hybrid cryptographic systems leverage asymmetric methods for secure key establishment and symmetric methods for efficient data encryption, creating a balance between security and performance. This dual approach forms the basis for most modern secure communication systems, especially in cloud computing and real-time applications where both data integrity and computational efficiency are critical.

Diffie–Hellman Key Exchange

The Diffie–Hellman key exchange is a foundational public-key mechanism designed to allow two parties to establish a shared secret over an insecure communication channel without prior arrangements. Its core purpose is to solve the challenge of securely generating encryption keys in open networks by enabling participants to derive a common secret value without directly transmitting it (Wagner, et al.).

In the classical mod p formulation, the protocol operates as follows:

1. **Public Parameters:**
The parties agree on a large prime p and a generator g of a multiplicative subgroup modulo p . These values are public and do not compromise security.
2. **Private Exponents:**
Each party independently chooses a private exponent: Alice selects a , and Bob selects b . These values are kept secret.
3. **Public Values Exchange:**
Alice computes her public value:

$$A = g^a \bmod p$$

and Bob computes his public value:

$$B = g^b \bmod p$$

They exchange A and B over the insecure channel.

4. Shared Secret Computation:

Using their private exponent and the other party's public value, both participants compute the shared secret:

$$\text{Alice: } S = B^a \bmod p = g^{ab} \bmod p$$

$$\text{Bob: } S = A^b \bmod p = g^{ab} \bmod p$$

Because $g^{ab} \equiv g^{ba} \bmod p$, both parties derive an identical secret S without transmitting it directly (Stallings, 2017).

This shared secret can then be used as an encryption key after proper processing through a key derivation or hashing step. Its confidentiality is ensured as long as the private exponents remain secret, and the discrete logarithm problem is computationally infeasible to solve in the chosen prime group (Wagner, et al.).

The security of Diffie–Hellman relies on the computational hardness of the discrete logarithm problem: an eavesdropper who observes p , g , A , and B cannot feasibly compute S without knowing the private exponents a or b (Wagner, et al.). When implemented properly with sufficiently large prime moduli or elliptic curves, the shared secret provides robust confidentiality against passive attackers, making it highly suitable as the foundation for secure symmetric encryption.

Key Derivation Using SHA-256

Why Convert the Raw Shared Secret

The raw shared secret produced by the Diffie–Hellman (DH) protocol should not be used directly as a session key, because it does not meet the requirements of modern cryptographic primitives. Its bit-length may not match the key sizes required by encryption or authentication algorithms, and the security of the majority of its bits is not guaranteed. Furthermore, the raw DH secret preserves algebraic structure from the underlying group, which could be exploited by attacks that take advantage of mathematical relationships between keys. A key derivation function (KDF) is necessary to produce a fixed-length, uniformly distributed key suitable for symmetric cryptographic operations, ensuring key independence and resistance to known-session-key attacks (Stiglic, 2001).

The Role of SHA-256 in Key Production

The cryptographic hash function SHA-256 is used as a simple Key Derivation Function (KDF) to make the final key secure and correct.

- **Fixed Length:** The most important role of SHA-256 is that it always produces an output of exactly 256 bits, no matter what the length of the input (Lindell, 2020). This provides the perfect length required for the AES-256 key.
- **Cleaning the Key:** SHA-256 "cleans up" the raw secret by destroying any mathematical properties left over from the DH calculation. This ensures that the final 256-bit key is statistically random and has high entropy, which is critical for strong security (Stallings, 2017).

AES-CBC Symmetric Encryption

Overview and Encryption Mechanism

The Advanced Encryption Standard in Cipher Block Chaining (CBC) mode is a symmetric encryption method that processes data blocks in a sequence. This mode is typically used for 128-bit blocks of data (Dworkin, December 2001).

According to the NIST SP 800-38A (Dworkin, December 2001) standard, the encryption process uses a chaining mechanism. For each plaintext block (P_i), two steps happen before the final ciphertext is created:

1. First, the plaintext block (P_i) is combined with the previous ciphertext block (C_{i-1}) using the XOR function.
2. Then, this result is encrypted using the secret key (K).

This sequential process creates a dependency chain (Stallings, 2017). It means that a small change in even one bit in an early block will lead to a complete change in all the following ciphertext blocks. This important security feature is known as the Avalanche Effect.

The mathematical operation for this process is:

$$C_i = E_k(P_i \oplus C_{i-1})$$

The Role of the Initialization Vector (IV)

The Initialization Vector (IV) is very important for the security of CBC mode. Since the first block of plaintext (P_1) has no previous ciphertext block (C_0), the IV is used as the initial input for the first XOR operation.

The main role of the IV is to make the encryption probabilistic (random) instead of deterministic (Lindell, 2020). The IV must be random and unique for every encryption operation, even if the same key is used. This random input ensures that if the same message is encrypted multiple times, the output ciphertext will be different every time. This unique starting point is essential to prevent Chosen-Plaintext Attacks (Niels Ferguson, 2010).

Why AES-CBC is Widely Used to Secure Stored Information

AES-CBC is widely used for securing Data at Rest, such as hard drives and databases, because it effectively hides statistical patterns in the data.

Unlike the simpler Electronic Codebook (ECB) mode, which would show if the same data blocks were repeated (Pelzl, 2010), the chaining mechanism ensures that identical plaintext blocks result in very different ciphertext (Stallings, 2017). This provides a high level of semantic security. Furthermore, CBC is a well-studied and reliable mode, making it standard-compliant for many applications, including the encryption of large files using necessary padding standards to fit the required block size (NIST SP 800-38A) (Dworkin, December 2001).

Real-World Applications

TLS/HTTPS Protocol

The Transport Layer Security (TLS) protocol, which is the basis for HTTPS, is a primary example of hybrid encryption.

- **Key Exchange (TLS Handshake):** During the handshake, modern systems use mechanisms like ECDH (Elliptic Curve Diffie-Hellman) to create a new session key for every connection. This provides Forward Secrecy, meaning past traffic cannot be decrypted even if the long-term key is compromised (Rescorla, August 2018).
- **Symmetric Encryption:** The established session key is used to encrypt the actual HTTP data flow using strong algorithms such as AES-256 in GCM mode, which offers both confidentiality and integrity protection (Stallings, 2017).

VPN Protocols (IPsec)

VPN protocols, such as IPsec, use a hybrid model to ensure the confidentiality of IP packets transferred through a secure tunnel.

- **Key Exchange (IKE):** The Internet Key Exchange (IKE) protocol, which relies on Diffie-Hellman, is used to securely create the necessary security associations and dynamic session keys for the tunnel. This process is documented in RFC 7296 (Charlie Kaufman, October 2014).
- **Symmetric Encryption:** Once the session keys are established via IKE, they are used to encrypt all data packets inside the VPN tunnel, ensuring data privacy while traveling across the public internet (Pelzl, 2010).

Encrypted Messaging Applications

End-to-End Encrypted messaging applications rely on a highly evolved hybrid model that continuously updates keys for stronger security.

- **Key Exchange and Renewal:** These systems often use Diffie-Hellman based schemes, like the Double Ratchet Algorithm, to constantly generate new, ephemeral session keys for nearly every message. This strong renewal mechanism significantly strengthens Forward Secrecy (Lindell, 2020).
- **Symmetric Encryption:** These rapidly changing session keys are used to encrypt the content of the message using powerful symmetric algorithms, such as AES-256 (Stallings, 2017).

Password-Vault Technologies

Password vaults use symmetric encryption heavily but replace network-based key exchange with robust Key Derivation from a master password.

- **Key Derivation (KDFs):** The system uses Key Derivation Functions (KDFs) like PBKDF2 or Argon2 to securely generate a strong symmetric encryption

key from the user's master password. KDFs are designed to be intentionally slow, which helps resist brute-force attacks (Lindell, 2020).

- Symmetric Encryption: This derived key is then used to encrypt the entire vault database using a secure symmetric mode like AES-256-GCM, ensuring that the stored data is protected even if the database file is compromised (Pelzl, 2010).

Conclusion

In conclusion, hybrid cryptography elegantly combines the best of both worlds: the secure key-exchange capabilities of asymmetric algorithms such as Diffie–Hellman with the speed and efficiency of symmetric encryption like AES-CBC. By deriving a shared secret over an insecure channel and then transforming it via a key-derivation function (KDF), such as SHA-256, into a fixed-length symmetric key, the system ensures both confidentiality and performance. This architecture underpins many of today's secure communication protocols from secure web browsing (TLS/HTTPS) to VPNs, encrypted messaging, and password-vault technologies. The hybrid model strikes a critical balance between strong security guarantees and practical efficiency, making it especially suitable for real-world applications where data security, speed, and resource constraints coexist. As such, hybrid cryptography remains central to modern secure communications and is likely to continue evolving as the backbone of trustworthy, efficient systems.

References

Charlie Kaufman, P. E. (October 2014). *Internet Key Exchange Protocol Version 2 (IKEv2) RFC 7296*. Internet Engineering Task Force (IETF).

Deokar, R. (2022). *File Transfer on Cloud using Diffie-Hellman Key Exchange in Conjunction with AES Encryption*.

Dworkin, M. (December 2001). *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*. NIST National Institute of Standards and Technology .

Lindell, J. K. (2020). *Introduction to Modern Cryptography*. CRC Press.

Niels Ferguson, B. S. (2010). *Cryptography Engineering Design Principles and Practical Applications*. Wiley.

Pelzl, C. P. (2010). *Understanding Cryptography A Textbook for Students and Practitioners*. Springer.

Rescorla, E. (August 2018). *The Transport Layer Security (TLS) Protocol Version 1.3*. Internet Engineering Task Force (IETF).

Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice 7th ed*. Pearson.

Stiglic, A. (2001). *Security Issues in the Diffie-Hellman Key Agreement*.

Wagner, D., Law, A., Shakir, F., Ngai, N., Kao, P., & Weaver, N. (n.d.). CS 161: *Computer Security*. Retrieved from <https://textbook.cs161.org/crypto/key-exchange.html>